Отчёт по лабораторной работе №5

Имитационное моделирование

Брыляков Никита Евгеньевич

Содержание

1 Теоретическое введение **5**
1.1 1. Сети Петри 5
1.2 2. Задача «Обедающие философы» 6
1.3 3. Моделирование на сетях Петри 7

2 Задание **9**

3 Цель работы **10**

4 Выполнение лабораторной работы **11**
4.1 Код модели 11
4.2 Базовые эксперименты 11
4.3 Анимация процесса 13
4.4 Итоговый отчёт 14

5 Выводы **16**

Список иллюстраций

4.1 Проверка структуры проекта 11
4.2 Код модели 11
4.3 Базовый эксперимент 12
4.4 Базовый эксперимент 12
4.5 Создание производных форматов 12
4.6 Запуск файла в jupyter-notebook 12
4.7 Анимация процесса 13
4.8 Анимация процесса 13
4.9 Создание производных форматов 13
4.10 Запуск файла в jupyter-notebook 14
4.11 Итоговый отчёт 14
4.12 Итоговый отчёт 14
4.13 Создание производных форматов 15

Список таблиц

1 Теоретическое введение

1.1 1. Сети Петри

Сеть Петри — это математический аппарат для моделирования дискретных систем, особенно параллельных, асинхронных и распределённых. Графически сеть Петри представляется как двудольный ориентированный граф двух типов вершин:

- **Позиции (places)** — пассивные элементы, обозначаемые кружками. Они описывают состояние системы (например, наличие ресурса). Внутри позиции могут находиться **фишки (tokens)** — неотрицательные целые числа, указывающие количество ресурсов.
- **Переходы (transitions)** — активные элементы, обозначаемые прямоугольниками. Они моделируют события или действия, которые могут произойти.
- **Дуги (arcs)** — направленные соединения между позициями и переходами. Входные дуги указывают, сколько фишек необходимо для срабатывания перехода; выходные — сколько фишек появится после срабатывания.

1.1.1 Правило срабатывания перехода

Переход разрешён, если каждая его входная позиция содержит не менее фишек, чем кратность входной дуги. При срабатывании из входных позиций удаляется указанное количество фишек, а в выходные — добавляется.

1.1.2 Формальное определение

Сеть Петри — это четвёрка $N = (P, T, F, M_0)$, где: - P — конечное множество позиций; - T — конечное множество переходов; - $F \subseteq (P \times T) \cup (T \times P)$ — множество дуг; - $M_0: P \to \{N\}$ — начальная маркировка.

1.1.3 Основные свойства

- **Ограниченность** — количество фишек в позиции не превышает заданного предела.
- **Активность** — для любого перехода существует достижимая маркировка, в которой он может сработать (отсутствие вечных блокировок).
- **Достижимость** — возможность из начальной маркировки попасть в желаемую.
- **Сохраняемость** — взвешенная сумма фишек постоянна.

1.1.4 Расширения

Временные, стохастические, раскрашенные и иерархические сети Петри.

1.2 2. Задача «Обедающие философы»

Сформулирована Эдгером Дейкстрой в 1965 году для иллюстрации проблем синхронизации в параллельных системах.

1.2.1 Классическая постановка

За круглым столом сидят N философов (обычно 5). Перед каждым — тарелка с едой, между соседями лежит одна вилка (всего N вилок). Каждый философ может: - думать, - голодать (хочет есть), - есть.

Чтобы поесть, философу нужны обе вилки — левая и правая. Если вилки заняты, он ждёт. Поев, философ кладёт вилки обратно.

1.2.2 Проблема взаимной блокировки (deadlock)

Если все философы одновременно возьмут левую вилку, каждый будет ждать правую, занятую соседом. В результате никто не может есть — система замирает.

1.2.3 Решение (арбитр)

Вводится дополнительный процесс (официант), который разрешает есть не более чем (N-1) философам одновременно. Это предотвращает deadlock.

1.3 3. Моделирование на сетях Петри

Каждый философ и каждая вилка представляются позициями, переходы соответствуют действиям: «взять левую вилку», «взять правую вилку», «начать есть», «положить вилки». Сеть Петри наглядно показывает возникновение deadlock и позволяет экспериментировать с механизмами синхронизации (например, арбитром).

1.3.1 Методы анализа

- **Детерминированное моделирование (ODE)** — описывает непрерывную динамику.
- **Стохастическое моделирование (алгоритм Гиллеспи)** — учитывает случайность времени срабатывания переходов.
- **Обнаружение deadlock** — проверка, есть ли в последней маркировке хотя бы один разрешённый переход.

- **Визуализация и анимация** — помогают наблюдать эволюцию маркировки во времени.

2 Задание

- Создать рабочий каталог для всего курса.
- Создать рабочее пространство для программ в рамках лабораторной работы.
- Выполнить все задания по тексту лабораторной работы.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
  - чистый код;
  - jupyter notebook;
  - документацию в формате Quarto.
  - Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Добавить в код в литературном стиле вычисление для набора параметров.
- Сгенерировать из литературного кода с параметрами:
  - чистый код;
  - jupyter notebook;
  - документацию в формате Quarto.
- Выполнить код из jupyter notebook с параметрами.
- Интегрировать документацию с параметрами в формате Quarto в отчёт.

3 Цель работы

Цели данной лабораторной работы:

- Построить сеть Петри для пяти философов, моделируя захват и освобождение вилок.
- Обнаружить состояние взаимной блокировки (deadlock), когда каждый философ взял одну вилку и ждёт вторую.
- Провести имитационное моделирование (стохастическое и детерминированное) и выявить наличие deadlock.
- Модифицировать сеть, чтобы предотвратить deadlock.
- Проанализировать результаты и оформить отчёт с графиками и анимацией.

4 Выполнение лабораторной работы

Предварительно проверим правильность структуры нашего проекта (рис. 4.1).

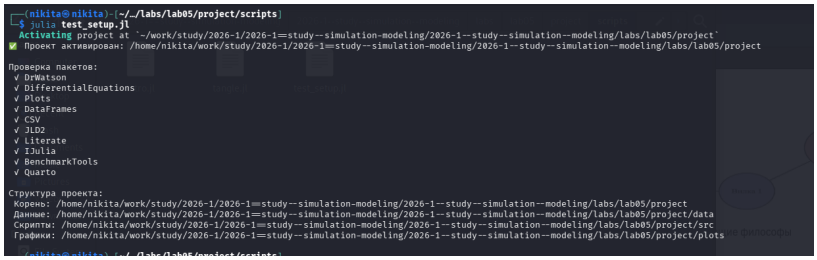

Рисунок 4.1: Проверка структуры проекта

4.1 Код модели

Создадим файл src/DiningPhilosophers.jl с описанием базовой модели сети Петри (рис. 4.2).

Рисунок 4.2: Код модели

4.2 Базовые эксперименты

Создадим файл scripts/dining_philosophers.jl. Скрипт выполняет основное моделирование и сравнение двух вариантов сети Петри: классическую

модель (без арбитра), в которой возможна взаимная блокировка (deadlock) и модифицированную модель с арбитром, которая должна предотвращать deadlock (рис. 4.3).

Рисунок 4.3: Базовый эксперимент

Запустим скрипт (рис. 4.4).

Рисунок 4.4: Базовый эксперимент

Создадим производные форматы с помощью скрипта tangle.jl (рис. 4.5).

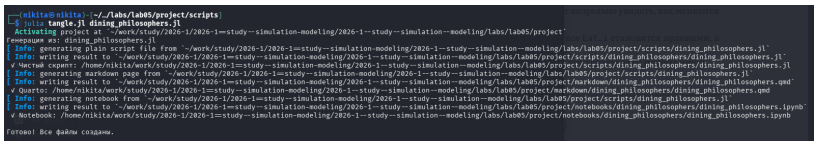

Рисунок 4.5: Создание производных форматов

Запустим файл ipynb в jupyter-notebook (рис. 4.6).

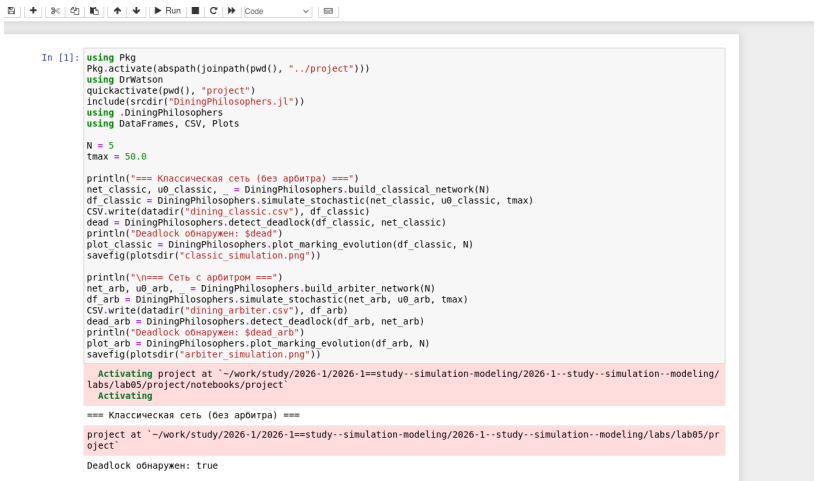

Рисунок 4.6: Запуск файла в jupyter-notebook

4.3 Анимация процесса

Создадим файл scripts/dining_philosophers_animation.jl. Анимация позволяет увидеть, как меняется маркировка (фишки) в каждой позиции, и особенно наглядно показывает возникновение deadlock в классической модели (рис. 4.7).

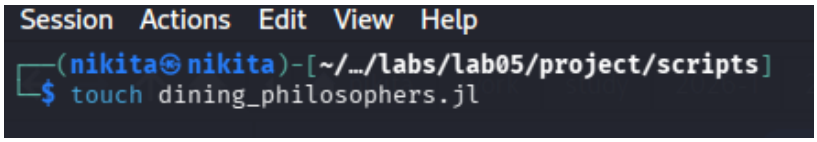

Рисунок 4.7: Анимация процесса

Запустим скрипт (рис. 4.8).

Рисунок 4.8: Анимация процесса

Создадим производные форматы с помощью скрипта tangle.jl (рис. 4.9).

Рисунок 4.9: Создание производных форматов

Запустим файл ipynb в jupyter-notebook (рис. 4.10).

```julia
using Pkg
Pkg.activate(abspath(joinpath(pwd(), "../project")))
using DrWatson
quickactivate(pwd(), "project")
include(srcdir("DiningPhilosophers.jl"))
using .DiningPhilosophers
using Plots, Random

N = 3
tmax = 30.0
net, u0, names = DiningPhilosophers.build_classical_network(N)

Random.seed!(123)
df = DiningPhilosophers.simulate_stochastic(net, u0, tmax)

anim = @animate for row in eachrow(df)
    u = [row[col] for col in propertynames(row) if col != :time]
        bar(
            1:length(u),
            u,
            legend = false,
            ylims = (0, maximum(u0) + 1),
            xlabel = "Позиция",
            ylabel = "Фишки",
            title = "Время = $(round(row.time, digits=2))",
            )
        xticks!(1:length(u), string.(names), rotation = 45)
    end

    gif(anim, plotsdir("philosophers_simulation.gif"), fps = 2)
    println("Анимация сохранена в plots/philosophers_simulation.gif")
```

```
  Activating project at `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation--modeling/labs/lab05/project/notebooks/project`
  Activating project at `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation--modeling/labs/lab05/project`
[ Info: Saved animation to /home/nikita/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation--modeling/labs/lab05/project/plots/philosophers_simulation.gif

Анимация сохранена в plots/philosophers_simulation.gif
```

Рисунок 4.10: Запуск файла в jupyter-notebook

Просмотрим результирующий gif-файл.

4.4 Итоговый отчёт

Создадим файл scripts/dining_philosophers_report.jl. Он нужен для сравнительного анализа двух моделей (с арбитром и без) по одному ключевому показателю — числу философов, находящихся в состоянии «Ест» (Eat_i). Скрипт генерирует сводный график (рис. 4.11).

```
┌──(nikita㉿nikita)-[~/…/labs/lab05/project/scripts]
└─$ touch dining_philosophers_report.jl
```

Рисунок 4.11: Итоговый отчёт

Запустим скрипт (рис. 4.12).

```
┌──(nikita㉿nikita)-[~/…/labs/lab05/project/scripts]
└─$ julia tangle.jl dining_philosophers_report.jl
  Activating project at `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation--modeling/labs/lab05/project`
Генерация из: dining_philosophers_report.jl
[ Info: generating plain script file from `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation--modeling/labs/lab05/project/scripts/dining_philosophers_report.jl`
[ Info: writing result to `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation--modeling/labs/lab05/project/scripts/dining_philosophers_report/dining_philosophers_report.jl`
√ Чистый скрипт: /home/nikita/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation--modeling/labs/lab05/project/scripts/dining_philosophers_report/dining_philosophers_report.jl
[ Info: generating markdown page from `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation--modeling/labs/lab05/project/scripts/dining_philosophers_report.jl`
[ Info: writing result to `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation--modeling/labs/lab05/project/markdown/dining_philosophers_report/dining_philosophers_report.qmd`
√ Quarto: /home/nikita/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation--modeling/labs/lab05/project/markdown/dining_philosophers_report/dining_philosophers_report.qmd
[ Info: generating notebook from `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation--modeling/labs/lab05/project/scripts/dining_philosophers_report.jl`
[ Info: writing result to `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation--modeling/labs/lab05/project/notebooks/dining_philosophers_report/dining_philosophers_report.ipynb`
√ Notebook: /home/nikita/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation--modeling/labs/lab05/project/notebooks/dining_philosophers_report/dining_philosophers_report.ipynb

Готово! Все файлы созданы.
```

Рисунок 4.12: Итоговый отчёт

Создадим производные форматы с помощью скрипта tangle.jl (рис. 4.13).

```
df_classic = CSV.read(datadir("dining_classic.csv"), DataFrame)
df_arbiter = CSV.read(datadir("dining_arbiter.csv"), DataFrame)
N = 5
```

```
  Activating project at `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation--modeling/
labs/lab05/project/notebooks/project`
  Activating project at `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation--modeling/
labs/lab05/project`
```

Out[1]: 5

Столбцы для состояния "Ест"

In [2]:
```
eat_cols = [Symbol("Eat_$i") for i = 1:N]

    p1 = plot(
        df_classic.time,
        Matrix(df_classic[:, eat_cols]),
        label = ["Ф $i" for i = 1:N],
            xlabel = "Время",
            ylabel = "Ест (1/0)",
            title = "Классическая сеть",
            )
    p2 = plot(
        df_arbiter.time,
        Matrix(df_arbiter[:, eat_cols]),
        label = ["Ф $i" for i = 1:N],
            xlabel = "Время",
            ylabel = "Ест (1/0)",
            title = "Сеть с арбитром",
            )
    p_final = plot(p1, p2, layout = (2, 1), size = (800, 600))
    savefig(plotsdir("final_report.png"))

    println("Отчёт сохранён в plots/final_report.png")
```

```
Отчёт сохранён в plots/final_report.png
```

Рисунок 4.13: Создание производных форматов

5 Выводы

В ходе данной лабораторной работы мной были построены классическая и модифицированная сети Петри для задачи «Обедающие философы», проведено стохастическое и детерминированное имитационное моделирование, обнаружен deadlock в классической сети и продемонстрировано его предотвращение с помощью арбитра, а также выполнен анализ результатов с визуализацией в виде графиков и анимации.